

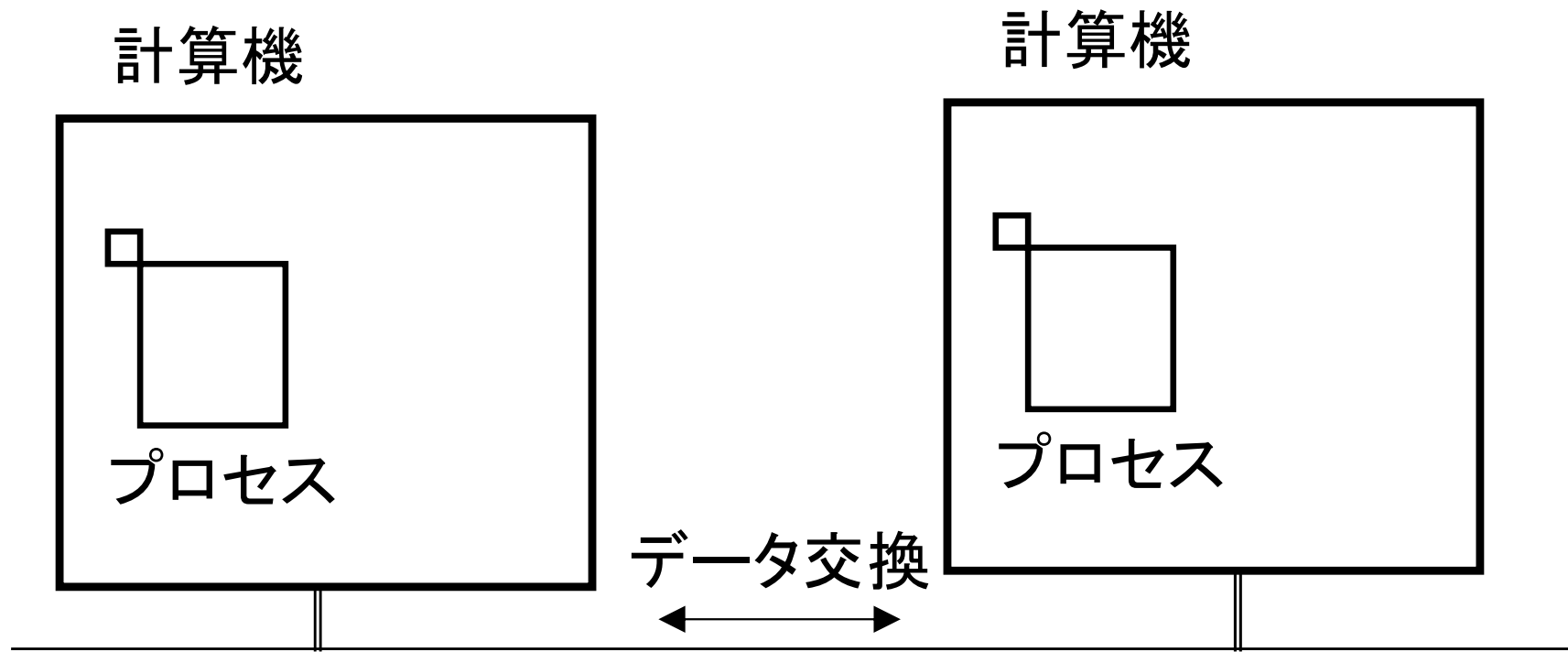
並列分散処理 (ネットワークシステム) 基盤技術

山内 利宏

岡山大学 工学部 工学科
情報・電気・数理データサイエンス系
情報工学コース

プロセスとメッセージ

- プロセス, または計算機をノードと呼ぶ
- 交換されるデータをメッセージと呼ぶ



2.1 通信プリミティブ

- メッセージを送受信するライブラリ関数やシステムコール関数を通信プリミティブと呼ぶ
 - send
 - receive
- 信頼性がある
 - 送信したメッセージは、途中で失われない
 - 有限時間以内に通信相手のプロセスに届けられる
 - ◆ そうでない場合、エラーメッセージが返ってくる
 - 途中でメッセージが失われると再送される
- 信頼性がない
 - 途中でメッセージが失われても、何もしない

結合（コネクション）

- 結合が必要な場合
 - 事前に結合を確立する必要あり
- 結合が不要な場合
 - 送りたいデータをすぐに送れる
- 例
 - 結合が必要な場合：電話
 - 結合が不要な場合：郵便

アドレス指定の方法

- 直接
 - 受信するプロセスを指定する
 - 例: UNIXのシグナル
- 間接
 - メールボックスを指定する
 - 複数のプロセスで共有もできる
- 現在利用可能な通信プリミティブでは, ほとんどが間接指定を利用

同期式，非同期式

■ 同期式とは

- sendでは，相手がreceiveをするまで処理が停止
- receiveでは，メッセージが届くまで，処理はブロック

■ 非同期式とは

- sendでは，メッセージのバッファリングが行われ，相手がreceiveしない状態でも，処理を先に進めることが可能
- バッファリングとは，OSなどが持っているメモリにメッセージをコピーすること
- receiveでは，メッセージが届いていなくても，即座に処理が戻ってくる
- メッセージの到着は，別にポーリングや割り込み，または別のプリミティブで調べる必要がある

ユニキャスト, マルチキャスト, ブロードキャストとは

- ユニキャスト
 - 1対1の通信(送信側プロセス1個, 受信側プロセス1個)
- マルチキャスト
 - 送信側プロセス1個に対し, 受信側プロセスが複数の場合の通信
- ブロードキャスト
 - 特定の物理的な通信媒体に接続しているプロセス(コンピュータ)すべてにメッセージを送ること
 - ハードウェアの機能が必須
- マルチキャスト通信は, 受信したいと思っているプロセスにだけ, メッセージが送られる

帯域保証と遅延保証

■ 帯域保証

- 毎秒どのくらいのデータを送ることができるかを保証すること
- 動画は、一定以上の帯域が保証されないと、コマ落ちが発生

■ 遅延保証

- sendしてからreceiveするまで、最長どのくらいの時間がかかるかを保証すること
- 電話では、衛星中継のように遅延が大きいと、会話ができない

データグラムとストリーム

■ データグラム

- 結合が作られず, ひとかたまりのメッセージを送受信
- sendの引数には, メッセージと通信相手のアドレスを与える
- 信頼性がない
- メッセージの順序が入れ替わることもある

■ ストリーム

- TCPで提供されている
- 通信する二つのプロセスの間で結合が作られる
- 信頼性がある
- メッセージの順序は入れ替わらない

通信プリミティブの性質

	TCP	IP	UDP	イーサネット	電話	郵便
send	非同期	非同期	非同期	非同期	非同期	非同期
receive	同期	非同期	同期	非同期	同期	非同期
信頼性	あり	なし	なし	なし	あり	なし
アドレス指定	間接	間接	間接	間接	間接	直接
結合	あり	なし	なし	なし	あり	なし
方向	双方向	単方向	単方向	単方向	双方向	単方向
マルチキャスト	不可	可能	可能	可能	可能	不可
帯域保証	なし	なし	なし	なし	あり	なし

15ページ, 表2.1

2.2 整列化

■ 整列化

- メモリ上に散らばって保存されているデータを、ネットワークに流すときに都合がよいように、**ひとかたまりにまとめること**
- ポインタを相手の計算機に送っても、ポインタが指すその先のデータを送らなければ、意味がない
- ポインタが多段になると整列化が複雑に
- Javaでは、シリアライズと呼ぶ

■ 非整列化

- 受け手側で、元(整列化前)のデータに戻すこと

バイトオーダー

- 分散プログラムでは、メッセージを送信するプロセスと受信するプロセスが異なる場合がある
- したがって、整数を送る場合でもバイトオーダーに気をつける必要がある
- リトルエンディアン
 - 下位番地に下位バイトを置く方法
 - 下位数8ビット, 16ビットだけが必要な場合有利
- ビッグエンディアン
 - 下位番地に上位バイトを置く方法
 - 上位バイトからアクセスする必要があるとき有利

整数・文字列・構造体・配列の送受信

- 異なるバイトオーダーを持つコンピュータで整数を送受信するとき、考えられる方法
 - 送信側、または受信側で相手に合わせて変換する
 - ◆ バイトオーダーが同じ場合には、何もしない
 - 標準的なバイトオーダー(ネットワークバイトオーダー)を定める
 - ◆ 送信側では、ネットワークにデータを流すときには、常に自分自身のバイトオーダー(ホストバイトオーダー)をネットワークバイトオーダーに変換する
 - ◆ 受信側では、ネットワークバイトオーダーから自分のホストバイトオーダーに変換する
- 現在のネットワークバイトオーダーとして、ビッグエンディアンが使われている(TCP/IPのIPアドレス, ポート番号)

バイトオーダを変換するライブラリ関数

名前	方向	ビット数
htonl()	ホストからネットワークへ変換	32ビット
htons()	ホストからネットワークへ変換	16ビット
ntohl()	ネットワークからホストへ変換	32ビット
ntohs()	ネットワークからホストへ変換	16ビット

h: ホスト

n: ネットワーク

l: long(32ビット)

s: short(16ビット)

0x12345678 htonl() 0x78563412

リトルエンディアンのホストの場合、バイトの入れ替えを行う
一方、ビッグエンディアンのホストの場合、何もしない

文字コード

- 送信側と受信側で文字コードが異なることがある
 - 例えば, ASCIIとUnicode
 - 常に文字コードの変換が可能とは限らない
 - ◆ 例: JISコードとUnicodeの間に変換できない文字
 - 複数バイトの整数を用いる文字コードでは, バイトオーダの問題が生じる
- 構造体や配列を整列化する
 - SunRPCでは, 自動的に整列化/非整列化を行うプログラムを生成できる
- 整列化の方法として, 文字列に変換する方法
 - Lisp言語のS式, CSV形式, 単純な文字列など

2.3 認証とアクセス制御

- 認証とは、情報の正当性や完全性を確保する技術
 - 利用者認証： アクセスしている人物が本人であるか確認すること
 - ◆ パスワードなどが使われる
 - ◆ 2要素認証がよく使われる
 - 指紋認証, ワンタイムパスワードなど
 - デジタル署名： 送られたメッセージが利用者本人が出したものかを確認する
- アクセス制御とは、ある利用者が、あるオブジェクトをいろいろな方法でアクセスするとき、どんなアクセスの方法なら許されるかを決定し、それを強制的に守らせること

(参考) NIST SP 800-63B-4: パスワードセキュリティの新基準 (1/2)

□ 従来の「常識」を覆す新要件

■ 定期的なパスワード変更の強制禁止

- 侵害の証拠がある場合のみ変更を要求すべきです
- 理由: 頻繁な変更は、ユーザーが予測可能で弱いパスワードを選択する原因になります

■ 複雑性要件の押し付け禁止

- 理由: このような要件は、ユーザーが「Password1!」のような予測可能なパターンを使用する原因になります

■ パスワードヒントの禁止

- これらは攻撃者にとって貴重な情報源となります。というか、これって複数の場所が知っている情報である確率が高く、したがって、こんなもので認証手段をリセットできたら目も当てられません。

- 引用元: <https://www.sakimura.org/2025/10/7710/>
-

(参考) NIST SP 800-63B-4: パスワードセキュリティの新基準 (2/2)

□ 推奨される要件

■ パスワードの長さ

- 単一要素認証: 最低15文字
- 多要素認証の一部: 最低8文字

■ ブロックリストの実装

- 検証者は以下を含むブロックリストと照合する必要があります:
 - 過去の侵害で漏洩したパスワード
 - 辞書に載っている単語
 - サービス名やユーザー名などの文脈固有の情報
 - 「123456」や「password」などの一般的なパスワード

■ パスワードマネージャーの積極的サポート

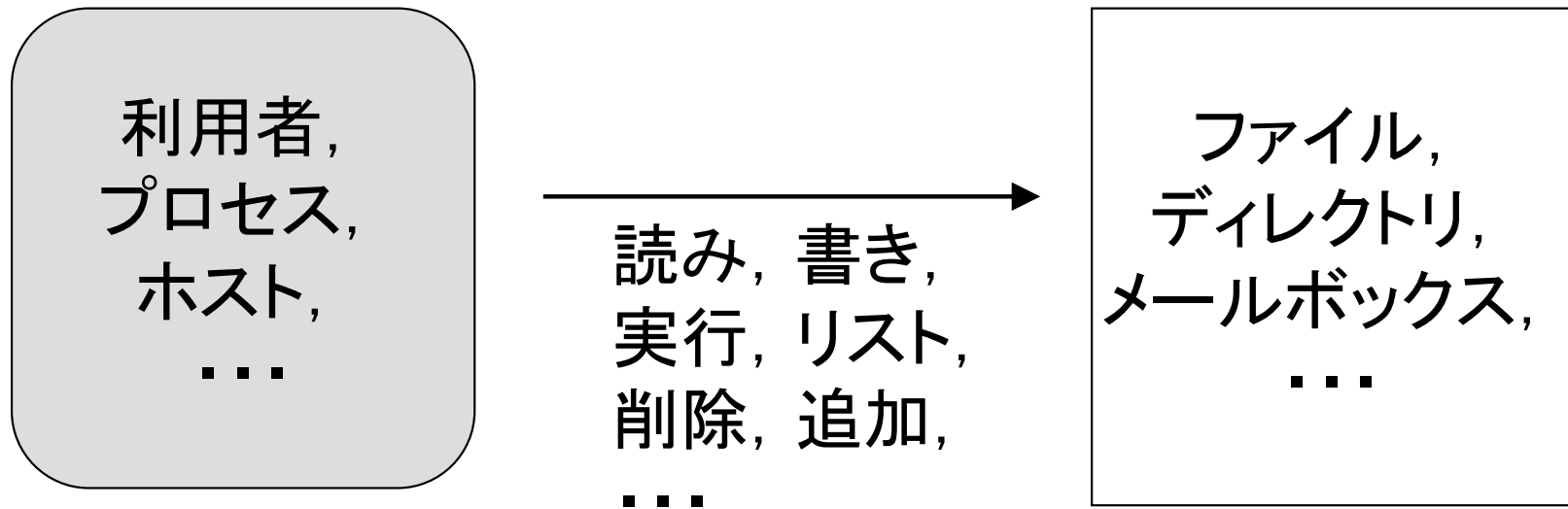
- 理由: パスワードマネージャーを使えば、ユーザーは強力で一意なパスワードを各サービスで使用できます
 - NOTE: それだけでなく、多くのパスワードマネージャーは、投入先のURLを検証しています。これはフィッシング耐性という意味で重要です。

- 引用元: <https://www.sakimura.org/2025/10/7710/>
-

アクセス制御

- 三つの概念
 - 主体(サブジェクト)
 - ◆ 利用者, またはプロセス
 - ◆ 分散処理では, ホストが主体になることがある
 - 客体(オブジェクト)
 - ◆ ファイルやディレクトリなど, OSレベルのもの
 - ◆ メールボックスやURLなど, APLレベルのもの
 - 操作
 - ◆ 読み, 書き, 実行
 - ◆ その他に, 印刷, リストの表示, 削除, 追加など, 独自の操作を考えることができる

アクセス制御のモデル



利用者認証

- 通信の前に結合を確立する通信プリミティブ
 - コネクションを確立した直後に1回
 - ◆ POP, APOP, telnet, ftpなど
- World Wide Web
 - 基本的には, 1ページを送受信するたびに結合を破棄
 - Basic認証では, 一つのページを得るたびに, クライアントは利用者名とパスワードをサーバに送信
 - クッキーを用いる場合もある
 - ◆ サーバは, 一度認証が済んだクライアントに対して, クッキーを設定する
 - ◆ サーバは, クッキーに含まれている要素を, 認証された利用者からのものとして扱う

実際のシステムの設定

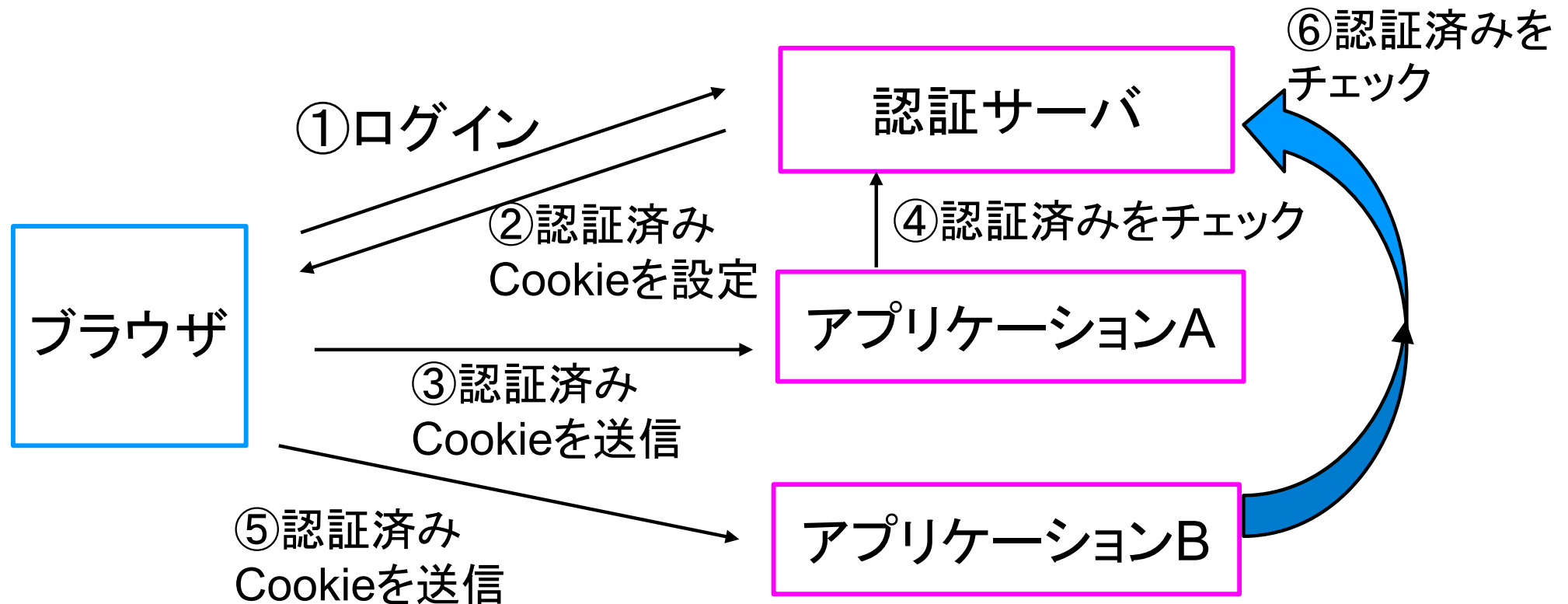
- 実際のシステムでは、オブジェクトの方に主体と許される操作の組のリストを持たせることが多い
 - UNIXでは、主体を所有者、グループ、その他に分類
 - 許可された操作のリストをファイルの属性として保持
 - Windows NT以降では、ACLを採用
 - ◆ 主体と許される操作、主体と許されない操作のリストをファイルに付加している
 - Apache Webサーバ
 - ◆ ページやURLをオブジェクトとして、オブジェクト毎に主体とそれに許された操作を記述する
 - ◆ 他にクライアントのホスト名やIPアドレスを利用できる

シングルサインオン(SSO)

- 一つのログインで複数のアプリケーションやサービスに対してアクセス可能になる特性や機能
- 利用者のメリット
 - 1つのユーザIDとパスワードの組を覚えるだけ
 - 一度ログインするだけで複数のアプリケーションを利用可能
- 管理者側
 - パスワードを保持する箇所を1カ所に集約可能
 - ◆ アタックサーフェス(攻撃対象領域)を限定
 - 認証処理に多要素認証などを組み込み, 強度を向上
 - アプリケーション毎のパスワード忘れなどの問合せ減

参考図書: 認証と認可Keycloak入門, リックテレコム

Cookieを利用したSSO方式イメージ



同ドメインの場合, Cookieで実現可能
(Cookieは同ドメインで共有可能)

ケーパビリティ

- 主体にアクセス制御の設定を持たせる方法
 - 各プロセスは、オブジェクトと許された操作の組をいくつか持つ
 - ケーパビリティは、他のプロセスに渡すことができる性質を持つ
- Linuxの事例
 - バージョン2.2以降で、管理者の権限をいくつかのグループに分割(このグループをケーパビリティと呼ぶ)
 - グループ毎に権限を有効、無効に設定可能
 - 例
 - ◆ CAP_SYS_MODULE, CAP_SYS_NICE, CAP_SYS_TIMEなど

認証

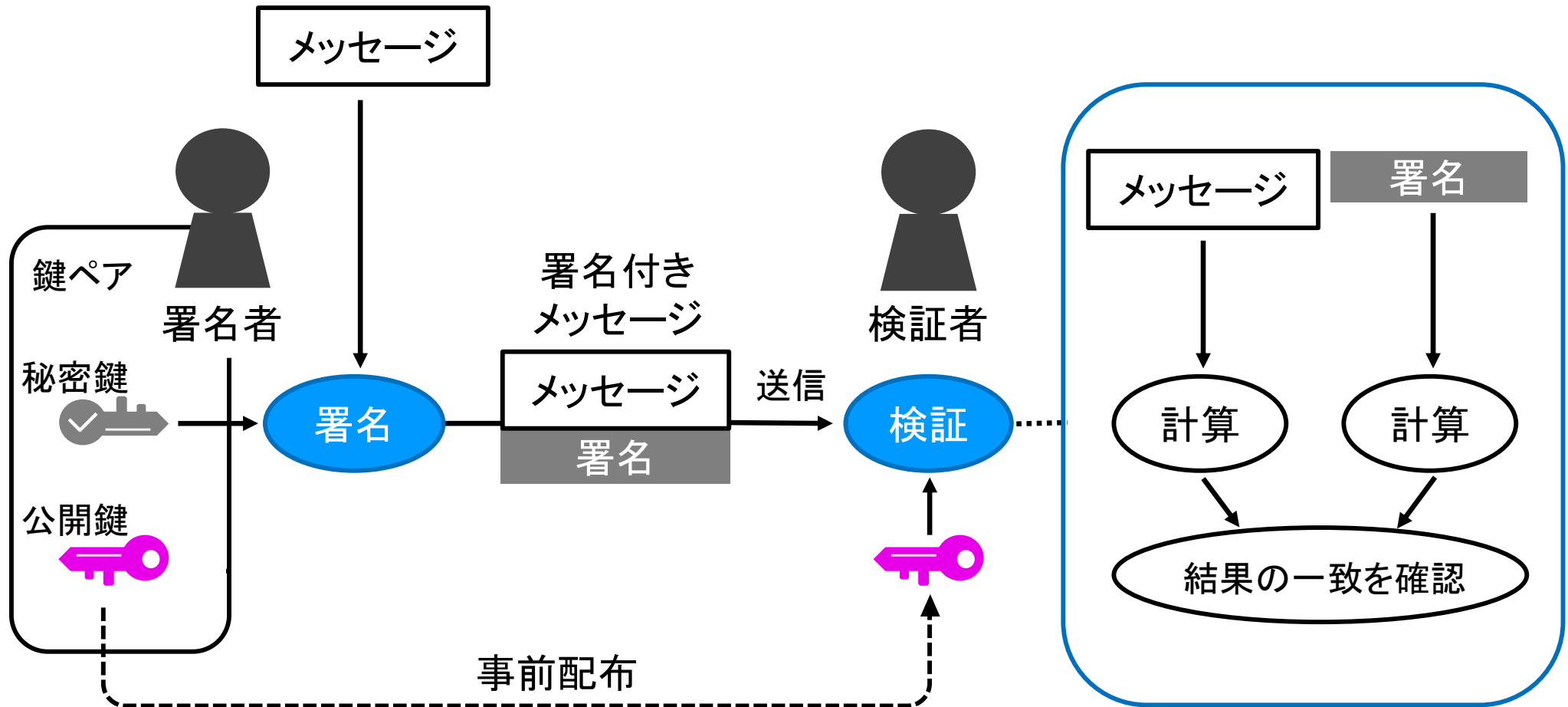
- 暗号による通信を行うときに必須
 - きちんと認証が行われなければ, man-in-the-middle 攻撃に対して脆弱となる
- Webサーバでは, 自分自身の証明書を提示し, ブラウザがその証明書を本物かどうか確認
 - 証明書の内容は, Webサーバの公開鍵とドメイン名で, 認証局によるデジタル署名が施されたもの
 - 認証局が信じられれば, その認証局により署名されたデジタル署名された内容も, 署名の検証により信頼可能
 - 検証により, Webサーバの公開鍵とドメイン名が信じられれば, そのWebサーバの秘密鍵でデジタル署名されたメッセージを受け取った段階で, Webサーバが本物とわかる
 - 参考: 教科書191ページ～194ページ

Webサーバとの通信の暗号化

- 最近では、SSL通信により、Webサーバとの通信が暗号化されることが推奨
 - SSLサーバ証明書により、通信相手のWebサーバが本物か認証可能
 - ◆ 認証局がWebサイトのドメイン、運営者の実在性などを審査
 - 通信データの暗号化
- SSL通信では、httpsで通信
 - ただし、最近ではフィッシングサイトも、httpsに対応しているので、鍵マークやhttpsでURLが始まるからといって、安全かどうかは判断不可

デジタル署名

SSL/TLS実践入門,
123ページ, 図2.14



例：署名にハッシュ値を含めておき、検証時にメッセージのハッシュ値と署名に含まれるハッシュ値を比較して検証

ハッシュ関数

■ 暗号学的ハッシュ関数

● 一方向性

- ◆ 関数の出力値から入力値を逆算もしくは推測することが著しく困難

- 暗号のように元の平文に復号できない

● ある特定の入力値に対するハッシュ値と全く同じハッシュ値を出力する別の入力値を求めることが著しく困難

- ◆ ある平文のハッシュ値が分かっている状態で、攻撃者が平文を改ざんして全く同じハッシュ値を作成するのは困難

● 衝突困難性

- ◆ 同じハッシュ値を出力する異なる2つの入力値を求めることが著しく困難

SSL証明書の種類

- ドメイン認証(DV)
 - ドメイン名の利用権
- 企業実在認証(OV)
 - ドメイン名の利用権
 - 組織の法的実在性
- EV認証
 - ドメイン名の利用権
 - 組織の法的・物理的実在性
 - 組織の運営, 承認者・署名者の確認

証明書の内容

- Webブラウザで見てください

岡山大学のサーバ証明書

証明書ビューア: www.okayama-u.ac.jp

全般(G)

詳細(D)

発行先

一般名 (CN)

組織 (O)

組織単位 (OU)

www.okayama-u.ac.jp

Okayama University

<証明書に含まれていません>

発行元

一般名 (CN)

組織 (O)

組織単位 (OU)

NII Open Domain CA - G7 RSA

SECOM Trust Systems CO.,LTD.

<証明書に含まれていません>

有効期間

発行日

有効期限

2023年8月3日木曜日 13:03:23

2024年9月2日月曜日 13:03:23

SHA-256 フィンガープリント

証明書

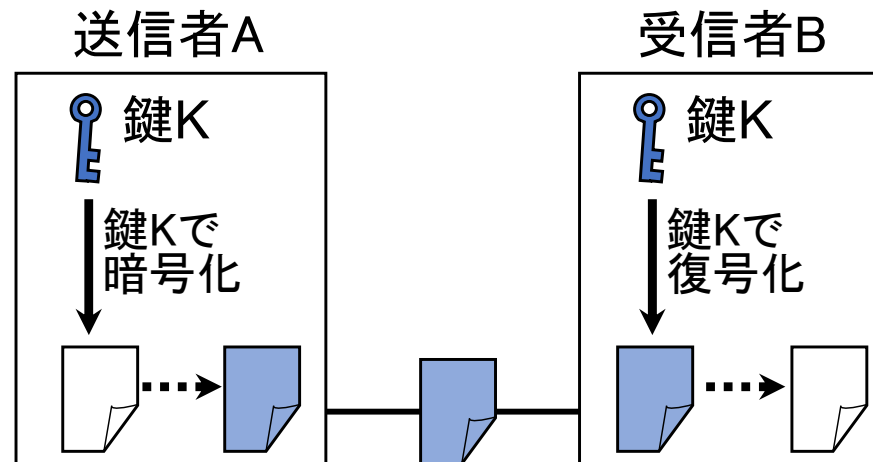
公開鍵

ea40d36a2bbece6c37ff1dcc706375b2302da523e2d809e1784cce
eb60fca4a9

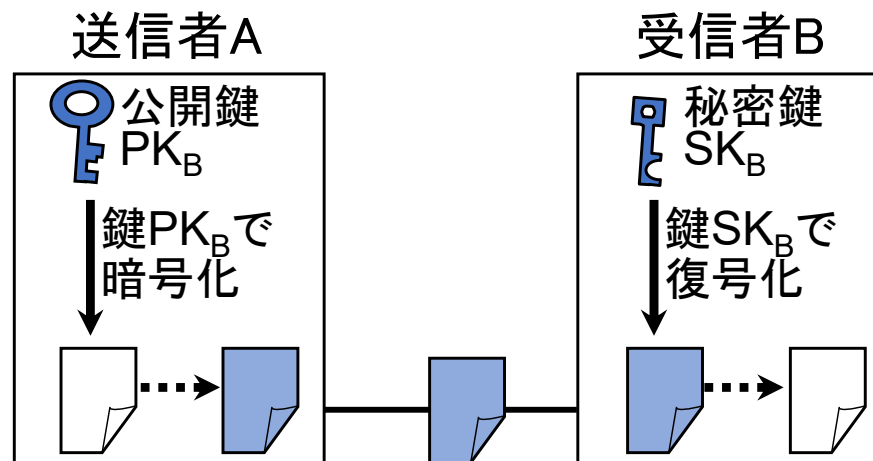
d5f51861cd156119e5c70102aacc41773a63b824d5ea4e9a13ac4c
84f2f74019

暗号化方式

分散処理,
191ページ,
図7.14



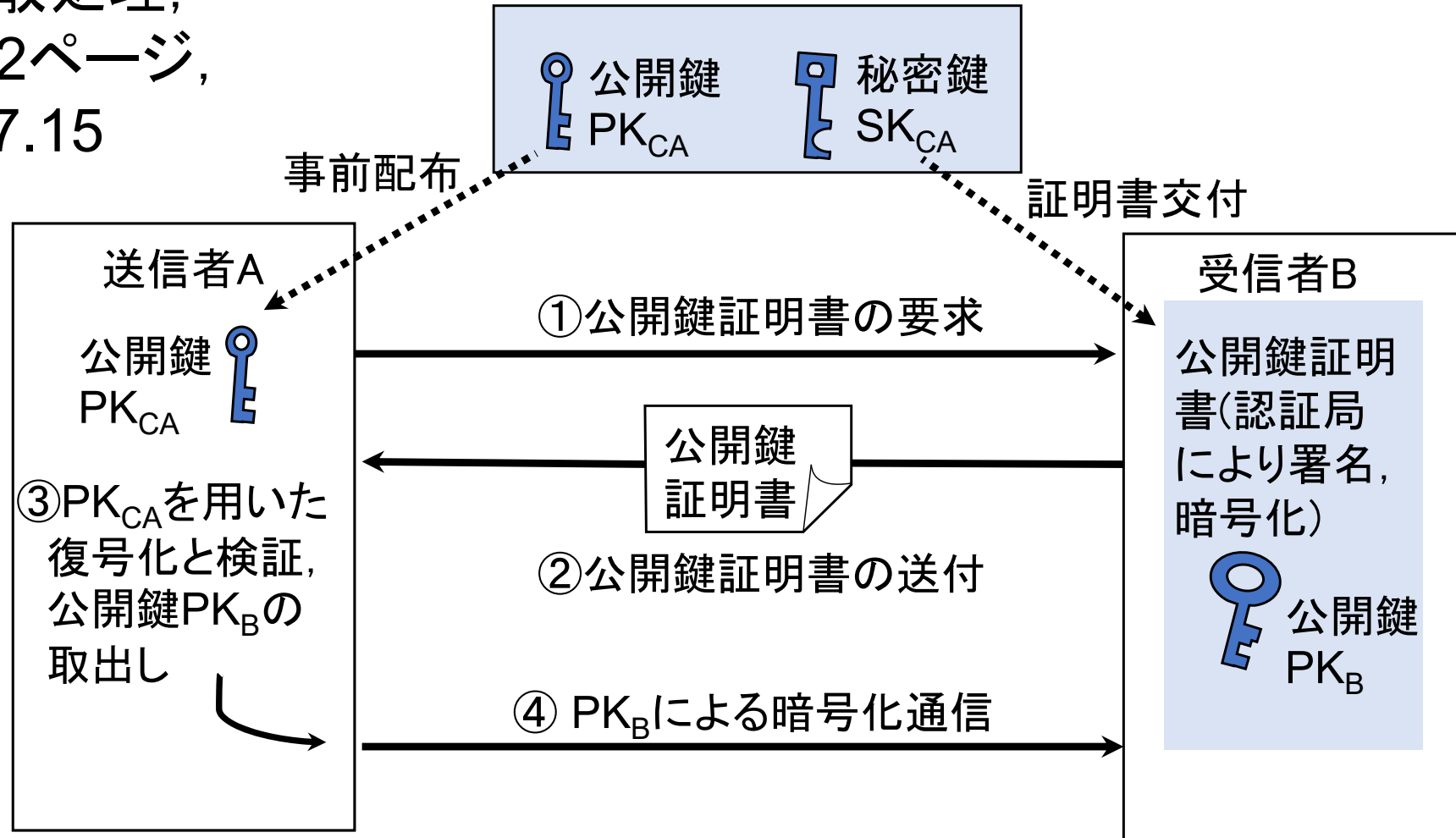
(a) 共通鍵暗号化方式



(b) 公開鍵暗号化方式

公開鍵証明書を用いた認証 (192ページ)

分散処理,
192ページ,
図7.15



SSLの概要(193ページ)



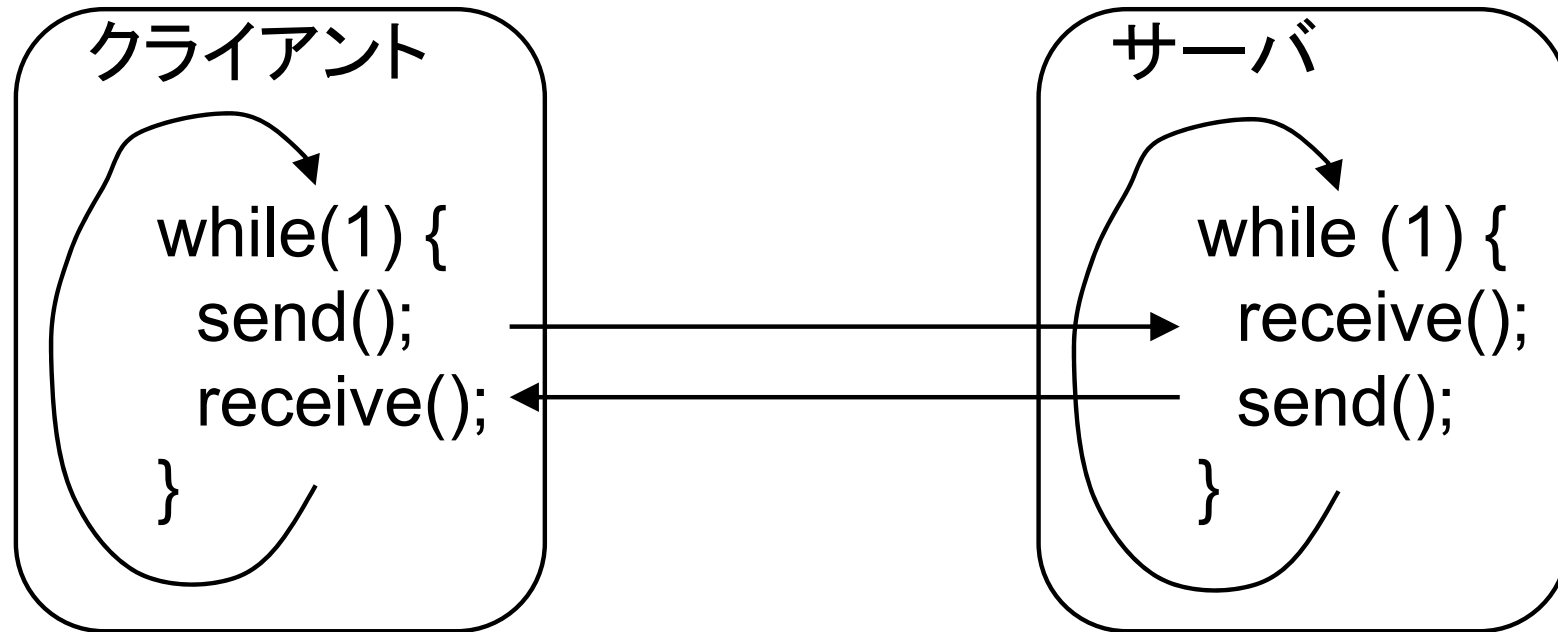
その他の認証方法

- 下位層で提供された機能を用いる方法
 - IPsec
 - ◆ IPパケット単位でデータの改竄防止や秘匿機能を提供するプロトコル
- SSL/TLSを使ってTCPのレベルと同等の通信
- SSHのトンネル機能
 - すでに構築されている暗号化されたストリームの中に、別のストリームを流す方法

2.4 クライアントサーバモデル

- 自由に分散プログラムを記述
 - プログラムが複雑になり, 見通しが悪い
 - バグも混入しやすくなる
- 意図的に制約を設定してプログラムを単純化
- クライアントサーバモデル
 - プロセス間通信を構造化したもの

通信パターン



- 次のようなプログラムは, 存在しない
 - sendしか行わない
 - receiveしか行わない
 - sendを2回してreceiveを1回だけ行う
 - receiveしたら, 処理の内容によって, sendしたりsendしなかったりする

クライアントサーバモデルの定義

- クライアント
 - サービスを受ける用のプロセス
 - 能動的
- サーバ
 - サービスを提供する方のプロセス
 - 多くの場合, 複数のクライアントにサービスを提供
 - 受動的
- 要求メッセージ: クライアントからサーバに送られる
- 応答メッセージ: サーバからクライアントに送られる

クライアントサーバの例

プログラム	通信パターン	サービス授受	結合作成
ファイルサーバ	サーバ	サーバ	サーバ*
DNS名前サーバ	サーバ	サーバ	サーバ*
Webサーバ	サーバ	サーバ	サーバ
Webブラウザ	クライアント	クライアント	クライアント
メールリーダ	クライアント	クライアント	クライアント
メール配送発信側	クライアント	クライアント	クライアント
メール配送着信側	サーバ	サーバ	サーバ
動画像送信	-	サーバ	サーバ*
動画像受信	-	クライアント	クライアント*

*UDPのように結合が作られない通信プリミティブが使われることがある

2.5 ソケットAPI

- TCP/IPをBSD系UNIXに導入するときに設計されたAPI
- システムコールに一般性を持たせ, どのプロトコルを使うかは最初にソケットオブジェクトを作成するときに指定する

ソケットAPIでのプロトコル指定

ドメイン	型	プロトコル
PF_INET	SOCK_STREAM	TCP (IPv4)
PF_INET	SOCK_DGRAM	UDP (IPv4)
PF_INET6	SOCK_STREAM	TCP (IPv6)
PF_INET6	SOCK_DGRAM	UDP (IPv6)
PF_UNIX	SOCK_STREAM	同一ホスト内のストリーム
PF_UNIX	SOCK_DGRAM	同一ホスト内のデータグラム

主要なシステムコールまたはライブラリ関数

名 前	説 明
socket ()	通信プロトコルに対応したソケットオブジェクトを作成する
connect ()	結合(connection)を確立させる. サーバのアドレスを固定する
listen ()	サーバ側で接続要求の待受けを開始する
accept ()	サーバ側で接続されたソケットを得る
bind ()	ソケットにアドレス(名前)を付ける
getpeername ()	通信相手のアドレス(名前)を得る
getsockname ()	自分のアドレス(名前)を得る
send (), sendto (), sendmsg ()	メッセージを送信する
recv (), recvfrom (), recvmsg ()	メッセージを受信する
shutdown ()	双方向の結合を部分的に切断する
getsockopt ()	オプションの現在の値を取得する
setsockopt ()	オプションを設定する
select (), poll ()	複数の入出力(通信を含む)を多重化する
write ()	メッセージを送信する. ファイルと共通
read ()	メッセージを受信する. ファイルと共通
close ()	ファイル記述子を閉じる. ほかに参照しているファイル記述子があれば, ソケットオブジェクトを削除する. ファイルと共通

引用元: 分散処理, オーム社, 31ページ, 表2.5

主要なIPアドレスを扱う関数

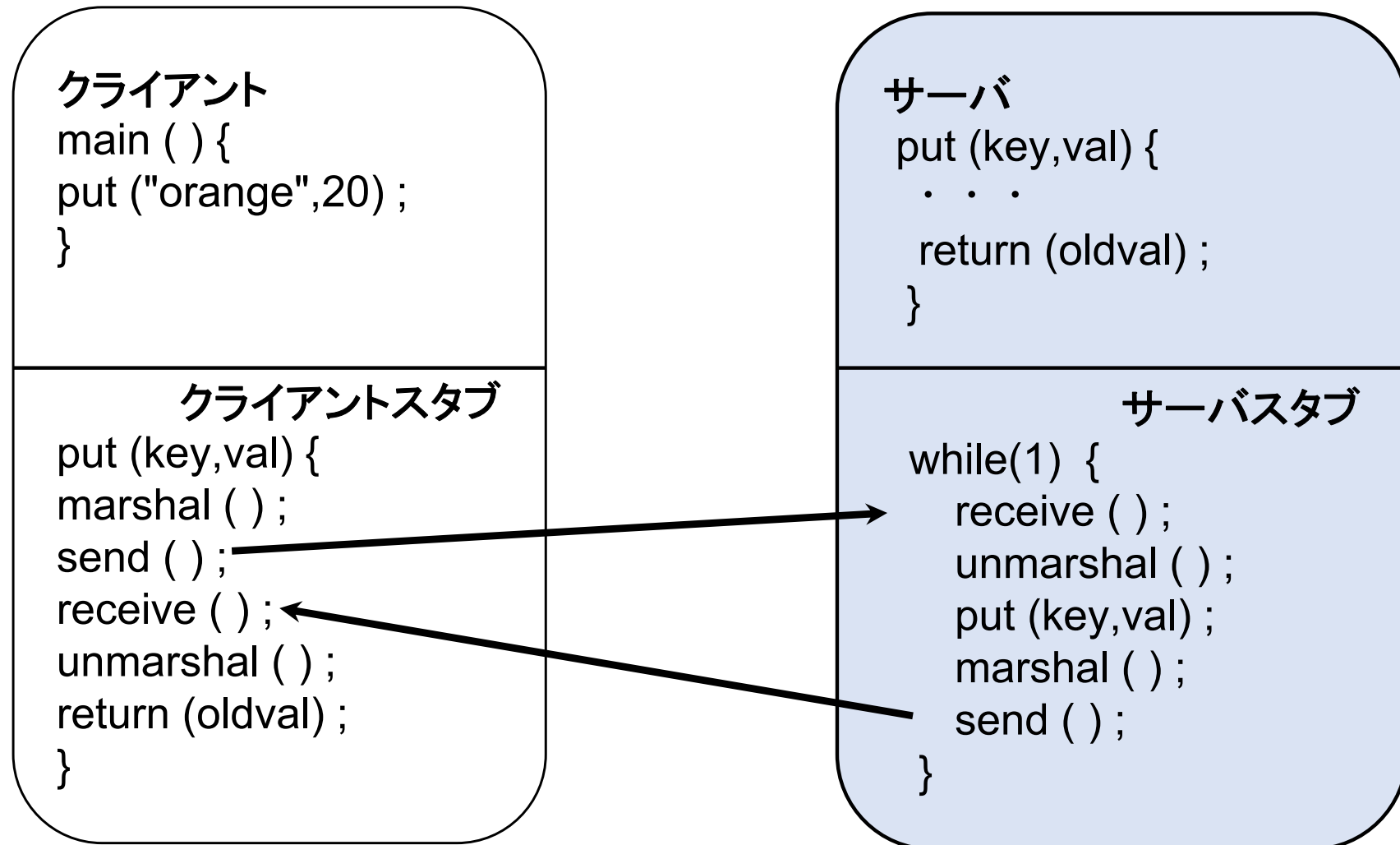
名 前	説 明
gethostbyname ()	ホスト名からIPアドレスを調べる
getaddrinfo ()	ホスト名からIPアドレスを調べる. IPv6対応
gethostbyaddr ()	IPアドレスからホスト名を調べる.
getnameinfo ()	IPアドレスからホスト名を調べる. IPv6対応
freeaddrinfo ()	getaddrinfo (), getnameinfo ()で得られた構造体を解放する

引用元:分散処理, オーム社, 32ページ, 表2.6

2.6 遠隔手続き呼び出し(RPC)

- 手続き呼び出しの形でプロセス間通信を行う方法
- RPCでは、プログラマは、直接sendやdo_operationなどの通信プリミティブを呼び出すプログラムを記述することはない
- クライアント側では、単に手続きを呼ぶプログラムを記述する
- サーバ側では、手続きの本体を記述する
- スタブにより、結びつけられる

スタブによる遠隔手続き呼出しの実現



引用元:分散処理, オーム社, 33ページ, 図2.11

スタブ

■ クライアント側スタブ

- 名前, 引数, 結果の型がインタフェースとなる手続きと同じ
- 内容: 引数の整列化, 要求メッセージ作成, 送信, サーバ側からの応答待ち, 応答メッセージの非整列化, 結果を返す

■ サーバ側スタブ

- 普段は, 要求メッセージを待っている
- 要求メッセージを受け取ると, 非整列化して引数を取り出し, 実際の手続きを呼び出す
- 呼び出した手続きの結果を整列化し, 応答メッセージとしてクライアント側に返す

ポインタの扱い

- 送受信できるデータは、基本的に値だけ
 - ポインタは送れない
- SunRPCでは、ポインタの先の1要素だけコピーして送る機能がある
 - ポインタによる単純なリストや木構造を送ることが可能
 - しかし、双方向リストなど、内部にループを含むものは送ることができない
 - ポインタで実現された有向非循環グラフを送ると木構造に展開される

バインディング

- クライアントとサーバの対応関係を、個々の手続き呼び出し前に決定すること
- サーバは複数のクライアントにサービスを提供する
- クライアントは、利用可能な複数のサーバから選ぶことができる
- SunRPC
 - バインディングは、サーバ側で動作しているポートマップというプログラムの助けを借りる
 - プログラム番号とバージョン番号という二つの整数でサーバを識別する
 - クライアントは、ポートマップにポート番号を問い合わせ、受け取ったポート番号に接続する

2.7 分散ファイルシステム

- 分散オペレーティングシステムの働きで作られるファイルシステム
 - 別のコンピュータ上のファイルでも、同一のコンピュータにあるように見せることが可能
 - ネットワーク上の別コンピュータで動作するプロセスであっても、ファイルを通じてデータを共有可能
→ 協調的な動作を行わせることが可能
- 分散透明性, ネットワーク透過性
 - ファイルの位置を意識させない性質
- ネットワークファイルシステム (NFS)
 - 分散ファイルシステムと異なり、サーバが故障するとそのサーバにあるファイルが利用できなくなる

NFS (1/3)

■ NFS

- Sun Microsystems社が開発したネットワークファイルシステム
- UNIX系OSでは、事実上の標準
- SunRPCを使って構築されている

■ NFS v2

- write()関数は同期書き込み
 - ◆ サーバがクラッシュしても何も失うものはない(無状態, stateless)
 - ◆ read()やwrite()の引数には、ファイルの中の読み書きする位置が含まれる

NFS (2/3)

■ NFS v2

- サーバ側では, UNIXシステムコールとは異なり, 読み書きする位置を状態として保持しない
- べき等
 - ◆ NFSのread()やwrite()は, 同じ引数で何度実行しても結果は同じ
- 無状態やべき等は, クラッシュからの回復を容易にしている
- ただし, write()で必ずディスク書き込む方法は, 性能面では不利

NFS (3/3)

■ NFS v3

- write()ではディスクに書き込まなくてもよい
- commit()という手続きを別途用意
 - ◆ データを確実にディスクへ保存
 - ◆ 複数のwrite()の結果をまとめて書き込み可能

■ NFS v4

- 複数のNFS手続きを1回の遠隔手続き呼び出しの中でまとめて実行可能
 - ◆ 通信遅延が大きいネットワークでも効率よく動作
- 利用者認証の機能を強化

SMBプロトコル/CIFSプロトコル

- Microsoft Windowsで用いられているネットワークファイルシステム
 - ファイル共有, 印刷など
 - クライアントサーバモデルに基づいて設計されている
- SMB/CIFSを実現したプログラムにSambaがある
 - Windowsパソコン間でファイルの共有可能
 - UNIXとWindowsの間でも共有可能

参考ページ: <https://learn.microsoft.com/ja-jp/windows/win32/fileio/microsoft-smb-protocol-and-cifs-protocol-overview>

分散共有メモリ

- ハードウェアで共有メモリがないシステムで、プロセス間の共有メモリを実現する機能
- 複数のノードに存在する複数のプロセスが、メモリを共有可能
 - 以下の点で通常の分散プログラムと異なる
 - ◆ データの交換方法
 - ◆ 同期の取り方
 - ◆ メッセージの紛失
 - ◆ ノードの部分的な故障